

Cordelia has wisdom: reshaping a live coding method

Jacopo Greco d’Alceo

`jacopo.greco dalceo@gmail.com`

Abstract. This paper reports on the reconstruction of *Cordelia*, a domain-specific language for performance (live coding), composition and notation based on Csound. Where the original implementation relied on hand-written, string-based mini-parsers to interpret its compositional syntax and translate into Csound language, the rewritten system is built on a formal Lark grammar processed by an Earley parser with a dynamic lexer and prioritised token classes, supporting LilyPond-style pitch notation alongside *Cordelia*’s own rhythmic and instrumental syntax. A seven-stage compiler pipeline replaces the previous single-pass conversion from text to Csound code, with a stage querying outward into a plugin-based *corpus*. The paper situates these changes within *Cordelia*’s founding principle of resource efficiency and argues that formalising the grammar has not constrained the language’s character but clarified it, making explicit a structure that was previously implicit.

Keywords: composition, live coding, method, Csound

1 Introduction

First introduced in previous work [3], *Cordelia*’s original formal environment proved insufficient to clarify the ideas it was meant to support. As observed in prior work on live coding, drawing on Green and Blackwell’s Cognitive Dimensions of Notations framework [4,2], live coding languages are particularly susceptible to *viscosity* — resistance to change or adapt to each personal vision. As *Cordelia* increasingly functioned as a compositional environment with branches and connections extending in every direction (i.e. Reaper script, Lilypond/Abjad outcomes), this resistance called for a return to essentials.

Reflections on reconsidering the idea of *pattern* pushed this inquiry toward a more conceptual and compositional question: where does live coding sit within compositional thinking? *Cordelia* turned toward isorhythm as a foundation, in which each parameter is read at its own independent rate based on its length.

Consider a talea of length 4 (`eu 4 8 in 8`, a euclidean rhythm of four onsets) paired with a colour of length 3 (`ff mp pp`, three dynamic levels). Because the two cycle independently, the pairing of rhythm-step to dynamic drifts across repetitions rather than repeating in lockstep: step 1 = (onset 1, *ff*), step 2 = (onset 2, *mp*), step 3 = (onset 3, *pp*), step 4 = (onset 4, *ff*) — the dynamic cycle having wrapped while the talea has not — step 5 = (onset 1 again, *mp*), and so on, the two cycles realigning only at their lowest common multiple (12 steps here). This offset is the isorhythmic effect: variation generated by cycle-length mismatch rather than by explicit notation of every combination.

It is nonetheless important to note that the fixed five-plus-one parameter structure of the earlier version no longer holds. Following the viscosity critique, *Cordelia*’s parameters are now open, notably through the addition of two further parameters: *cycle* and *character*. The purpose of this openness is an effort to *merge the syntax of composition and of performance into a single notation*.

Cordelia addresses and minimises viscosity by proposing an essential, minimalistic framework — one made possible by the intrinsic and exponentially expanding openness of Csound itself. This is achieved through:

- I. a formal grammar and staged compiler, replacing the original prototype;
- II. a naming vocabulary that extends the language’s literary self-understanding into its own source code;
- III. a session-state model for live continuity, which we call *wisdom*.

2 *Cordelia*’s pipeline

The original implementation parsed opcodes and parameters *positionally*, through regular expressions and string logic distributed across the codebase. Each new syntactic feature required its own ad hoc mini-parser, duplicating logic that properly belongs to a grammar and making the system increasingly difficult to extend without breaking existing behaviour.

The rewritten system instead defines a formal grammar processed by an Earley parser [1] with a dynamic lexer, implemented using the Lark parsing toolkit [5], allowing multiple notations to coexist with *Cordelia*’s own rhythmic and instrumental syntax within a single grammar, rather than across several independent parsers. The rewritten pipeline separates these concerns into seven discrete stages, grouped into four conceptual phases, as shown in Table 1.

Table 1. The seven pipeline stages, grouped into four conceptual phases.

Stage	Phase
lex	} grammar
parse	
transform	
deduce	} contextual
compare	
learn	wisdom
offer	} translate
	emit

Each stage performs a single, well-defined task. **lex** segments the incoming stream at @ into discrete poems and strips comments. **parse** matches each poem against the grammar via the Earley parser. **transform** converts the parse tree into Python objects (variables or instruments). **deduce** identifies each verse’s quality by matching against corpus keywords and manifests, contextually and without explicit type annotation. **compare** checks each current object against its state from the previous cycle, tagging it **patched/unpatched** and **init/release**. **learn** commits these compared states to the session’s persistent registry, so later phrases can reference prior pitches, qualities, and instrument identities directly. **offer** exposes this updated context back to the corpus’s plugin functions, so that deduction in the next cycle can draw on it — closing the loop between wisdom and deduce. **emit** translates the resolved objects into Csound code.

These seven stages group into four phases: *grammar* (lex, parse, transform) turns raw text into structured objects; *contextual* (deduce, compare) resolves those objects’ meaning against the corpus and prior session state; *wisdom* (learn, offer) commits that resolution to persistent memory and exposes it to the next cycle; *translate* (emit) turns the resolved objects into Csound code.

Each stage operates independently and communicates through well-defined intermediate structures, rather than the entangled state of the original prototype. Separating parsing from semantic resolution removes the duplicated mini-parsers of the earlier system and allows every stage to be reasoned about, and tested, on its own.

3 A Poetics of grammar

The basic unit of composition in *Cordelia* is delimited by the character @. This marks the point at which the first stage of the pipeline segments the incoming stream of code into discrete chunks, which the grammar then parses and the transformer converts into Python objects. Comments are introduced by ; (as in Csound), and their removal is likewise the responsibility of the lex stage.

Each such chunk is either a *variable* or an *instrument* — translated, respectively, into a global value or an instrument object during the transform stage. Both share a *header* — an @ followed by a name and an optional *voice id* (# followed by a number, distinguishing one instance of a given instrument from another) — which supplies the object’s identity and is later translated into a Csound

unique global variable (*gk*). An instrument may additionally carry any number of *modifiers* — UDOs that act as audio effects on its output. Within a verse, individual *items* are separated by white space alone; between successive verses, the median point `·` (or the pipe `|`, used interchangeably) is used, or a newline. An instrument requires a delimiter or newline before each of its verses; a variable does not.

These are the grammar’s production names as they operate at the parsing stage, kept pragmatic for maintainability. It is at the next stage — *transform*, converting matched text into Python objects — that *Cordelia*’s poetic vocabulary is applied: a header becomes an *epigraph*, an instrument becomes a *phrase*, a variable becomes a *statement*, a modifier becomes an *invocation*, and an item becomes a *foot*. The grammar itself reads as follows:

```
header:      "@" NAME voice_id?
voice_id:    "#" NUMBER

node:        instrument | variable
variable:    header _SPACE verse+
instrument:  header modifier* ((_DELIM | _NL) verse)+

modifier:    _NL* MODIFIER_KIND NAME array? ("{"|"(")?)

verse:       item (_SPACE item)*
item:        syllabe | repeat | expr
syllabe:     func | array | NOTE | NUMBER | NAME | ALEA | DEGREE
```

The language deliberately avoids commas as separators. This use of white space as the primary separator, together with the absence of case distinction in identifiers, is a deliberate constraint in service of writing speed during live performance.

Naming the transformed AST objects after units of verse is not merely an internal convenience. Since *Cordelia* treats notation itself as a philosophical problem, this naming is itself an act of notation about notation: the vocabulary the system settles into once code is resolved into meaning makes a claim about what a live-coded utterance *is*, positioning it as an inscribed line within a poem rather than as an instruction for execution.

4 Contextual deduction and wisdom

Cordelia, even in its earlier version, was built on the principle that code is always fully read rather than validated block by block. This removes the need to check each block before evaluating the next, favouring rapidity over the more conventional pattern of defining and validating discrete blocks in sequence. It also implies that each object maintains a single, continuous identity throughout a session — a premise that underlies the *wisdom* model introduced below, in which *Cordelia* retains awareness of every instrument currently playing, and can be written against that context directly. This also intrinsically resists the cue-list model of live coding performances.

Once an object is created — whether a variable or an instrument — the variable case simply produces a Csound global variable with a value inside an instrument. An instrument itself has three parts: a name, a set of modifiers (effects applied to its output), and a set of qualities. Qualities are the parameters of the underlying algorithm and are deduced through Python plugin functions: a quality may be identified by a keyword at the start of a token (i.e. *eu* for euclidean rhythms), or by matching against strings present in a corresponding JSON manifest. Every such deduction is governed by the same wisdom principle.

Once each quality has been deduced and identified, all current objects are compared against the previous session state to determine which are *patched* or *unpatched* — that is, whether a quality has changed or remained unchanged — and which are *init* (a freshly spawned object) or *release*. This comparison determines what is retained in, and what is removed from, the emitted code.

Through these successive passes, *Cordelia* accumulates *wisdom* of the full compositional situation: complete awareness of the state and of the pitches in use, such that both can be referenced directly from within the code.

Two consequences follow from this construction. First, an instrument’s quality need not be specified explicitly in the grammar: it is deduced contextually by *Cordelia*, allowing the compositional flow to proceed without imposing an ordering structure or requiring exhaustive specification. Second, qualities can be concatenated freely — any element may appear more than once, written directly one after another. This suggests that an intermediate language along these lines (tentatively, *isocordelia*) could approach the immediacy of notation itself, moving still closer to what we have elsewhere called an effort towards the instant: narrowing the gap between composition and performance.

5 The Corpus shell

We call the corpus a *shell* because it is *Cordelia*’s outer boundary with the outside world: the single point through which external material — instruments, tunings, samples, plugin functions — is admitted into the language, each retained in whatever native format it arrives in rather than being converted into a uniform internal representation.

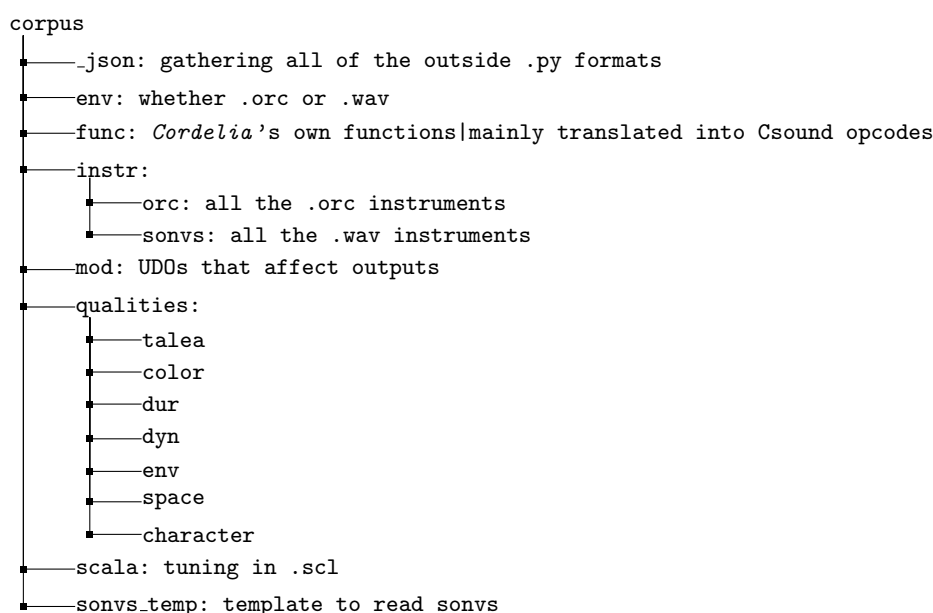


Fig. 1. Structure of the *corpus*.

The *corpus* (Fig. 1) is a fully hybrid system, written in Python, capable of accepting a multitude of source formats — *.orc* for instruments and modifiers, *.wav* for instruments read as samples or as envelopes, *.scl* for tuning systems, among others. Each format is treated according to its own conventions and is retained in its original form, in order to preserve compatibility and flexibility across contexts. Some formats pass through a scripting stage that indexes them into a *.json* manifest for rapid lookup within *Cordelia*; others — Python functions — are loaded directly as Python modules.

6 Exempla Gratia

```
@cordelia.eu 3 8 in 8.ff mp pp.cls^2 -cls.edo31 c. 1 3 7
```

This is a single *phrase*: the epigraph *@cordelia* names the instrument, and four verses (*qualities* in the instrument object) follow, separated by the median point *..*. Each verse is passed through the pipeline.

During *lex* and *parse*, the phrase is segmented at *@* and matched against the grammar, yielding one epigraph and four verses, each composed of one or more feet. During *transform*, these feet are converted into Python objects; during *deduce*, each verse’s *quality* is identified contextually, without any explicit type annotation in the source:

- `eu 3 8 in 8` is a *euclidean rhythm* verse, it gives the rhythm of the instrument;
- `ff mp pp` is deduced as a *dynamics* verse, its three feet describing a dynamic envelope from *fortissimo* through *mezzo-piano* to *pianissimo* — each of these dynamics will be used for each note event of the talea rhythmic section;
- `cls^2 -cls`, which becomes `cls cls -cls` is deduced as an *envelope* verse: the first foot repeats a value twice via the **REPEAT** token (`^2`), the last negation reverse the reading of the envelope;
- `edo31 c. 1 3 7` is deduced as a *pitch* verse, combining a tuning name with LilyPond-style duration notation.

7 Conclusion

This paper has traced the reconstruction of *Cordelia* from a functionally sufficient but increasingly viscous prototype into a formally grounded compositional environment. The formal grammar and staged pipeline do not merely replace ad hoc parsing with more rigorous engineering; they make explicit a structure — contextual deduction, patch/unpatch reconciliation, and the accumulation of *wisdom* across a session, that the earlier system enacted only implicitly, and therefore fragilely. In this sense the rebuild answers, rather than abandons, the founding provocation of *Cordelia*, now offering a grammar flexible enough to accommodate that same openness without collapsing under it.

The naming of the pipeline’s stages after units of verse, and the session-state model itself named *wisdom*, are not incidental to this argument. They continue *Cordelia*’s treatment of notation as inscription: a claim about what a live coded utterance *is*, carried into the source code of the compiler that processes it.

Several extensions are already under way atop this foundation, including finer-grained management of Csound function tables, per-section orchestra queuing, and a multi-threaded runtime coordinating scheduling, performance, and control input concurrently. More speculatively, the concatenative openness of the grammar suggests that a leaner dialect — *isocordelia*, could approach the immediacy of notation itself, further narrowing the gap between composition and performance that has motivated this project from its outset. Composition, in this view, remains what it has always been for *Cordelia*: a permanent process of individuation, in which the boundaries between language, notation, and performance stay deliberately fluid.

<https://github.com/jacopogreco/alceo/CORDELIA-REBORN>

References

1. John Aycock and R. Nigel Horspool. Practical Earley parsing. *The Computer Journal*, 45(6):620–630, 2002.
2. L. Church and T. Green. Cognitive dimensions - a short tutorial. In *Proceedings of 20th Psychology of Programming Interest Group 2008*, 2008.
3. Jacopo Greco d’Alceo. Cordelia, crafting a method while live coding in csound. In *Proceedings of the International Computer Music Conference*, pages 69–73, Wien, 2024.
4. Christopher Alex McLean. *Artist-Programmers and Programming Languages for the Arts*. PhD thesis, Goldsmiths, University of London, 2011.
5. Erez Shinan. Lark: A parsing toolkit for Python. <https://github.com/lark-parser/lark>, 2017.